JusticeFlow

Police Case & Evidence Management System

Multi-Course Integrated Project Proposal | DBMS · SDA · AI · OS

Executive Summary

JusticeFlow is a mission-critical Police Case & Evidence Management System that demonstrates measurable, independently verifiable contributions from four computer science disciplines. Each course solves a distinct, non-overlapping problem that naturally arises in legal-grade software.

Course	Core Responsibility	Key Contribution
DBMS	Data Integrity & Audit	Immutable audit logs, ACID transactions, stored procedures
SDA	Architecture & Patterns	5 design patterns, UML diagrams, layered architecture
AI	Intelligent Decision Support	Crime hotspot detection, case prioritization, workload balancing
OS	Process Isolation & Safety	Privilege daemon, file integrity monitor, process synchronization

System Overview

JusticeFlow manages the full investigation lifecycle — from FIR registration through evidence collection to case closure — enforcing legal-grade compliance at every step. Core entities include Officers (with rank hierarchy), Cases/FIRs, Evidence (with chain of custody), Criminals, and an immutable Audit Trail.

Primary workflows:

- FIR Registration → Investigation → Evidence Collection → Case Closure
- Officer Assignment & Duty Roster Management
- Evidence Chain of Custody with tamper detection
- Role-based authorization enforced at the OS process level
- AI-assisted case prioritization and patrol optimization

Module Details

1. DBMS — Data Integrity & Audit Foundation

The database layer is the non-negotiable core of JusticeFlow. It enforces legal-grade correctness through a fully normalized schema (3NF), ACID-compliant transactions, and automated auditing.

Key technical demonstrations:

- Immutable audit triggers that log every INSERT/UPDATE/DELETE to Audit_Log with old and new values
- Soft-delete pattern on Evidence (is_deleted flag) — evidence is never physically removed
- Stored procedures for business logic (e.g., assign_officer_to_case with rank and workload validation)
- Role-restricted views (e.g., constable_cases showing only assigned cases)
- Composite indexing on Cases(status, filed_date) with measurable before/after benchmarks

Verification	Run provided SQL test scripts: confirm audit trail completeness, referential integrity enforcement, transaction rollback safety, and query plan optimization.

2. SDA — Architecture & Design Patterns

The application layer is modeled with a complete UML suite and implemented using five design patterns, each solving a real architectural problem — not forced inclusions.

Pattern	Problem Solved
Strategy	Rank-conditional case status transitions (Inspector vs. SHO privileges)
Chain of Responsibility	Layered authorization: Constable → Inspector → SHO → Commissioner
Observer	Case updates simultaneously notify AuditLogger, NotificationService, AnalyticsPipeline
Factory	Unified report generation (daily summary, case history, chain-of-custody)
Singleton	Database connection pool — single shared resource, controlled access

UML deliverables: Use Case, Class (with inheritance hierarchy), Sequence (FIR filing, evidence addition), State (case lifecycle), and Activity diagrams.

Verification	Review UML diagrams for correctness; inspect C++ code for pattern implementation; verify open/closed principle — adding a new officer rank requires zero changes to existing pattern code.

3. AI — Intelligent Decision Support

Three ML agents run as separate read-only processes against the database. AI supports — never replaces — human authority. All predictions are explainable.

Agent	Algorithm	Output
Crime Hotspot Analyzer	DBSCAN (unsupervised clustering)	Heat map of top 5 crime zones with patrol recommendations
Case Priority Recommender	Random Forest (supervised)	Ranked case queue with SHAP-based feature explanations
Officer Workload Balancer	Hungarian Algorithm (optimization)	Optimal case-to-officer assignments minimizing workload variance

Training data: 5,000+ synthetic cases generated with realistic correlations; validated against UCI ML Repository crime datasets. AI results are written to a dedicated analytics schema and surfaced in the officer dashboard.

Verification	Run AI scripts independently; confirm hotspot precision > 70%, priority model accuracy > 75%; inspect analytics tables and SHAP explainability outputs.

4. OS — Process Isolation, Concurrency & System Auditing

The OS layer provides the infrastructure that all other layers depend on. We build system-level components rather than merely using OS features.

- Privilege Daemon (justice_authd): Root daemon on a Unix socket validates requesting process UID against officer rank before escalating to DB_ADMIN for sensitive operations — then immediately drops privileges (seteuid).
- File Integrity Monitor: Uses inotify to watch the evidence directory; any modification attempt triggers rollback from backup and logs a security alert.
- Process Synchronization: Named POSIX semaphores (sem_open) prevent race conditions when multiple officers update the same case concurrently.
- Job Scheduler: Custom mini-cron using SIGALRM and fork/exec runs audit log rotation (hourly), ML retraining (daily), and report generation (weekly).

- System Monitor: Reads /proc/stat and /proc/meminfo to expose real-time CPU, memory, and process health metrics.

Verification	Inspect daemon source; run concurrency test (50 simultaneous processes, 0 data corruption expected); attempt evidence file modification and confirm restoration; verify scheduled job logs.

Integration & Cross-Course Harmony

A single user action — filing a new FIR — exercises all four layers simultaneously:

Step	Layer	What Happens
1	OS	justice_authd authenticates officer's UID and enforces rank
2	SDA	CaseFactory creates the case object; AuthorizationChain validates filing rights
3	DBMS	Atomic transaction: INSERT into Cases, audit trigger fires, FK validated
4	AI	New case triggers hotspot re-analysis and priority scoring
5	Dashboard	Officer sees FIR number, crime zone classification, and priority score

Remove any layer and the system loses critical, non-replaceable functionality — the test of genuine integration.

Deliverables & Timeline

Sprint	Weeks	Milestone
1 — Foundation	1–2	Core DB schema, class diagrams, data collection strategy, daemon skeleton
2 — Core Features	3–4	Triggers & stored procedures, design patterns, first ML model, privilege escalation
3 — Integration	5–6	End-to-end workflow, AI dashboard, file integrity monitor, concurrency tests
4 — Polish & Demo	7–8	Performance tuning, documentation, demo scenarios, instructor test suites

Success Metrics

Course	Metric	Target
DBMS	Audit trail completeness	100% of critical changes logged
DBMS	Referential integrity	0 orphaned records in any test
SDA	Design patterns implemented	Minimum 5, all justified
SDA	Cyclomatic complexity	< 10 per function
AI	Hotspot detection precision	> 70%
AI	Priority model accuracy	> 75%
OS	Race condition prevention	0 data corruption in 50-process test
OS	Unauthorized access blocking	100% of attempts denied and logged

Technical Stack

Language: C++ (application layer, OS components) · Database: PostgreSQL · AI: Python with scikit-learn & NumPy · OS: Linux / POSIX APIs · IPC: Unix domain sockets, shared memory, named semaphores · UML: Lucidchart / Draw.io · Build: Makefile · Version Control: Git

Conclusion

JusticeFlow is a single coherent system where each course contributes an essential, independently demonstrable layer. The legal domain naturally demands all four disciplines — data correctness, architectural clarity, intelligent assistance, and OS-level safety — making every integration point authentic rather than artificial.